Module 1: Inheritance, Polymorphism, Abstract Classes & Interfaces

Problem Statement

Design a core hierarchy for a logistics system handling different types of transport vehicles.

Requirements

1. **Interface (GPSNavigable)**:
   - Declare method: void updateLocation(double lat, double lon).
   - Include a default method: default void pingServer() printing "Ping sent to central GPS server".
2. **Abstract Class (Vehicle)**:
   - Private fields: vehicleId (String), maxCapacityKg (double), currentFuelPercent (double).
   - Constructor to initialize vehicleId and maxCapacityKg.
   - Abstract method: public abstract double calculateEfficiency().
   - Concrete method: refuel(double amount) updating fuel percentage (cap at 100.0%).
3. **Concrete Subclasses**:
   - Truck: Extends Vehicle and implements GPSNavigable. Has additional field numberOfAxles (int).
     - Efficiency formula: $\text{Efficiency} = \frac{100}{\text{numberOfAxles}} \times \left(\frac{\text{currentFuelPercent}}{100}\right)$.
   - ElectricScooter: Extends Vehicle (does NOT implement GPSNavigable). Has additional field batteryHealth (int).
     - Efficiency formula: $\text{Efficiency} = \text{batteryHealth} \times 1.5$.
4. **Polymorphic Test Script (Main)**:
   - Create an array or array list of base type Vehicle containing both Truck and ElectricScooter instances.
   - Loop through the collection, invoke calculateEfficiency(), and use the instanceof pattern matching to update GPS coordinates only for objects implementing GPSNavigable.

Module 2: Exception Handling & Custom Exceptions

Problem Statement

Add robust validation and custom error handling for vehicle operations.

Requirements

1. **Custom Unchecked Exception (OverCapacityException)**:
   - Triggers when attempted cargo weight exceeds maxCapacityKg.
2. **Custom Checked Exception (InsufficientFuelException)**:
   - Triggers when attempting a trip that consumes more fuel than currently available.
3. **Implementation Details**:

    - Add method to Vehicle: void loadCargo(double weightKg) throws OverCapacityException.
    - Add method to Vehicle: void drive(double distanceKm) throws InsufficientFuelException.
        - *Fuel consumed = distanceKm * 0.15% per km.*
4. **Exception Handling Test**:
    - Write a driver program with a try-catch-finally block.
    - Simulate loading dynamic inputs (e.g., loading 5000kg into a 2000kg capacity scooter).
    - Explicitly catch OverCapacityException and InsufficientFuelException separately, log relevant context, and ensure resource teardown in a finally block.

Module 3: Java Collections Framework

Problem Statement

Manage a fleet registry, track delivery queues, and route driver assignments efficiently.

Requirements

1. **Fleet Management (List)**:
    - Use an ArrayList<Vehicle> to store all active vehicles.
    - Sort the list in descending order based on maxCapacityKg using a custom Comparator<Vehicle>.
2. **Package Dispatch Queue (Queue / Deque)**:
    - Implement a PriorityQueue<Package> where packages with higher urgency scores are processed first.
    - Class Package: fields packageId (String), weight (double), isPriority (boolean).
3. **Driver Assignment Mapping (Map)**:
    - Create a HashMap<String, Vehicle> mapping driverLicenseNumber to their assigned Vehicle.
    - Write a search lookup method to retrieve vehicle details given a driver's license ID.
4. **Unique Inspection Log (Set)**:
    - Store completed trip inspection IDs using a HashSet to guarantee no duplicate inspections are recorded.

Module 4: Generics

Problem Statement

Build a type-safe generic storage container for audit records and dynamic data tracking.

Requirements

1. **Generic Container (GenericAuditLog<T>)**:

- Define class GenericAuditLog<T> to store time-stamped log items of any type T.
- Internal data structure: List<T>.
- Methods: void addEntry(T item), T getLatestEntry(), List<T> getAllEntries().

2. **Bounded Generics**:
   - Write a utility method: public static <T Vehicle extends> double calculateTotalCapacity(List<T> vehicleList).
   - The method must accept any list containing objects that inherit from Vehicle (using upper-bounded wildcard ? extends Vehicle) and return the sum of maxCapacityKg.
3. **Generic Repository Interface**:
   - Define interface Repository<T, ID> with methods void save(T entity), T findById(ID id).
   - Implement a concrete VehicleRepository implementing Repository<Vehicle, String>.